C64 Visual Assembler — User Manual

Version 2.0.2

A visual, block-based 6502 assembler for the Commodore 64. Build programs by dragging and dropping instruction blocks, and see the generated assembly and machine code in real time.

Table of Contents

---

1. Interface Overview

The app is split into three main panels:

Panel	Description
Left — Palette	All available instruction and macro blocks. Search or browse by category.
Center — Program	Your program. Drag blocks here, reorder them, edit operands.
Right — Output	Live ASM view and/or memory monitor output.

---

2. Block Palette

The palette on the left lists all available blocks grouped by category:

- **Data movement** — LDA, LDX, STA, STX, ...
- **Arithmetic** — ADC, SBC, INC, DEC, CMP, ...
- **Logic** — AND, ORA, EOR, BIT
- **Jumps & Branches** — JMP, JSR, RTS, BNE, BEQ, ...
- **Register operations** — TAX, TAY, INX, DEX, ...

- **Shift & Rotate** — ASL, LSR, ROL, ROR
- **Stack** — PHA, PHP, PLA, PLP
- **System** — CLC, SEC, NOP, BRK, ...
- **Illegal instructions** — LAX, SAX, DCP, ...
- **Structure** — LABEL, COMMENT, REGION, ENDREGION
- **Macros** — LOOP, NEXT, FOR, ENDF, PUSH, PULL, TEXT, BYTE, WORD, FILL, ALIGN, STRING, DATA, RAWBYTES, RAWTEXT, PETSCII, INCBIN, SID, INCLUDE, TABLE, ORG, MACRO, ENDM, INVOKE, IF, ELSE, ENDIF, SPRITE_INIT, SPRITE_POS, WAIT_RASTER, JOYSTICK, MOUSE, SPRITE_COL, LOADFILE, REU_CHECK, REU_STASH, REU_FETCH, REU_SWAP, TURBO_SET, SUPERCPU_DETECT, TURBO_ENABLE

Use the **search box** at the top of the palette to filter by name. Click the **Add selected block** button or drag a block into the program area.

---

3. Program Area

- **Drag & drop** blocks from the palette, or **reorder** existing blocks by dragging their handle (≡).
- Each block shows its **mnemonic**, **operand field**, and **addressing mode selector** (where applicable).
- Click the ▸ **/** ▾ toggle to collapse or expand a block.
- Use the **× (delete)** button on a block to remove it.
- **Collapse All** button folds all blocks at once.

Operand input

- For branch/jump instructions (`BNE`, `JMP`, `JSR`, etc.) a **label picker** dropdown appears — click a defined label to insert it.
- Number format follows the **HEX / DEC** toggle in the toolbar (see section 5).

---

4. ASM View

The right panel shows the generated output in real time.

Output modes

Mode	Description
ASM	6502 assembly source with addresses and labels
Monitor	Hex / byte dump (C64 monitor style)
Disasm	Pure 6502 disassembly: address · hex bytes · mnemonics with resolved numeric operands. Macros are expanded to individual instructions (TEXT → LDA/STA pairs, LOOP → LDX, etc.). BYTE/WORD/FILL data shown as chunked hex dump. No macro names, comments, or annotations in output.
Both	ASM on top, monitor below
Disassembler	Same as Disasm — dedicated tab for the disassembly view

Mode	Description
Toolkit	C64 reference panel: 16-colour palette swatch + PETSCII control-code and printable-character cheat sheet. Read-only — see the "Toolkit tab" subsection below for details.
Options	Program settings panel — number format, macro source toggle, debugger params

Toolkit tab

The **Toolkit** tab in the ASM view is a read-only quick-reference panel — it never modifies your program. Two sections:

Section	Content
C64 colour palette	16-swatch grid showing every C64 colour with its index (0–15 / `$00` – `$0F`) and name. Click a swatch to copy its hex index to the clipboard. Hover for the colour name (Light Blue, Brown, etc.).
PETSCII control codes	Common control codes for `CHROUT` ($FFD2): colour change codes (`$05` white, `$1C` red, `$1E` green, `$1F` blue, ...), cursor movement (`$11` / `$1D` / `$91` / `$9D`), reverse on/off (`$12` / `$92`), `$93` clear screen, `$8E` / `$0E` charset switches. Also a printable-range cheat sheet (32-64 punctuation, 65-90 A-Z, 91-95 brackets, 96-127 graphics, 160-191 shifted graphics, 192-223 mirror).

The Toolkit is the fastest way to look up a colour index or a PETSCII control byte without leaving the editor.

Options tab

The **Options** tab contains the settings that affect code generation and output display:

- **Macro source** — when ON, macro definition blocks (MACRO...ENDM) show their source code inline in the ASM view.
- **Program start address** — now set via an **ORG block** in the program area rather than a separate input field. The first ORG block defines the program's load address; subsequent ORG blocks start additional sections at different addresses.
- **Debugger params** — three inline toggles controlling which flags are passed to the external debugger on launch:
  - `-jmp` **ON/OFF** — jump directly to the program's start address after loading.
  - `-unpause` **ON/OFF** — unpause the debugger immediately on load.
  - `-wait` **ms ON/OFF** — adds a `-wait <ms>` delay before unpausing; select 500 ms or 1000 ms from the dropdown.
- **Compile info** — shows a summary of the compiled program (code start address, size, BASIC SYS stub status).

Clicking an ASM line

Click any line in the ASM view to **highlight the corresponding block** in the program area.

ASM line numbers

The ASM panel displays **line numbers** (`001 |` , `002 |` , ...) to make troubleshooting easier when a compile error points to a specific line.

- The visual line numbers are for diagnostics only.
- **Copy ASM** still copies the clean source text **without** line-number prefixes.

Compile progress modal

During heavier actions, a centered progress modal appears with a progress bar:

- **Run in VICE** — compiling/building PRG and launching emulator.
- **Debug** — compiling/building PRG and launching debugger.
- **Import ASM** — parsing and materializing blocks from pasted source.

The modal closes automatically when the action completes or fails.

---

5. Settings & Toolbar

Control	Description
Number base (HEX / DEC / BIN)	Sets the display/input format for operands throughout the UI. BIN mode displays values as binary with `%` prefix (e.g. `%11111000`). The ASM view always shows each block in its own format.
Language	Switch between English and Hungarian
Theme	Light / Dark / OLED — select from the theme picker in the Settings menu. OLED uses a pure-black background for AMOLED displays.
CRT retro mode	Toggles a full-screen CRT filter: scanlines, phosphor vignette, flicker, and barrel distortion. State is saved between sessions.
BASIC SYS stub	Prepends a BASIC line that calls SYS to your program's origin
Sample	Load a built-in example program
Zoom in / out	Scale the block UI (affects all block elements)
Save Project	Save the current program as a `.json` project file
Load Project	Load a previously saved project
Open Project (`Menu → File`)	Open a multi-file `.proj` project and open all source files as tabs
Save Project (`Menu → File`)	Save the current `.proj` project (project panel must be open)
Close Project (`Menu → File`)	Close the currently open project and all its file tabs. Prompts to save unsaved changes. The project panel resets to its empty state.
Import ASM	Opens a paste dialog and imports textual 6502 ASM into blocks
Save PRG	Export the compiled binary as a `.prg` file
Run (split button)	The main ▶ **Run** button runs the current mode; click the ▾ arrow to switch between: **Run as PRG** (compile and launch VICE directly), **Run via D64** (package into a .d64 disk image and launch VICE), or **Run on hardware** (send PRG to a C64 Ultimate / 1541 Ultimate device). See Section 12 and Section 13.
Debug (RetroDebugger)	Compile and launch in RetroDebugger with breakpoints, symbols, and autostart flags (see Section 9)
Run with Exomizer	Checkbox in the Settings menu — when enabled, all Run and Build operations crunch the PRG through `exomizer sfx sys` before launching or saving. Works with Run as PRG, Run via D64, Run on Hardware, Build PRG, and Build D64. Configure the Exomizer executable in **Hardware Settings** first.

Control	Description
Hardware Settings	Open the hardware configuration dialog — configure VICE, Exomizer, RetroDebugger, and C64 Ultimate (host, password, connection test). See Section 13.
New program...	Opens a confirmation dialog, then clears all blocks from the program area
Collapse All	Collapse all blocks
About	Version info
What's New	Changelog
Knowledge Base	Reference links (6502 opcodes, C64 KERNAL, memory map, colors)
Check for Update	Open the itch.io page to check for a newer release

Import ASM (quick reference)

The Import dialog accepts common 6502 source patterns and converts them into blocks:

- `* = $1500` → ORG block
- `Label:` → LABEL block
- `Label: .byte 0` → LABEL + BYTE blocks
- `.byte ...` → BYTE block
- `; comment` (or inline `; ...`) → COMMENT block
- instructions (`lda`, `jsr`, `beq`, etc.) → instruction blocks with detected addressing mode

Import parsing notes and best practices

- Local labels like `.wait` are imported as standard labels (dot removed), and references are normalized accordingly.
- For `($zp),Y` / `($zp,X)` style addressing, use a concrete zero-page byte (`$FB`, `$FC`, etc.) for best compatibility.
- Avoid ambiguous short labels that look like hex (`cc1`, `dead`, `beef`) in branch contexts; prefer names like `loop_cc1`.
- If your program starts with data (`.byte`) before executable code, add an explicit entry jump (for example `JMP Start`) at the top.

6. Expert Mode

Expert Mode is a full-featured direct-text 6502 assembly editor that lives alongside the block editor. Every tab can be in either Block mode or Expert mode — you switch between them freely at any time using the **Block / Expert** toggle in the top bar.

Switching modes

- **Block → Expert:** the current program is serialised to text (one instruction per line, labels, macros as directives). Edits in Expert mode are synced back to the block array whenever you switch back or trigger an action.
- **Expert → Block:** the text is parsed with `parseAsmText()` and the result replaces the block program. A compile-error dialog is shown if parsing fails.
- **Blank lines** are preserved through round-trips: empty lines in the Expert editor appear as thin dashed spacers in Block mode and are restored as empty lines when switching back to Expert.

Editor layout

```
┌──────────────────────────────────────────────────────┐
│ [toolbar]  Block │ Expert ← tab toggle                │
├────────────┬────────────────────────────┬────────────┤
│  Palette   │   ASM text editor          │  Disasm    │
│  (opt.)    │   (monospace, editable)    │  panel     │
│            │                            │  (opt.)    │
└────────────┴────────────────────────────┴────────────┘
```

Panel	Toggle	Description
Palette	`#expert-palette-btn`	The left block palette — drag blocks into the editor or click to insert at cursor
ASM editor	always visible	Full monospace textarea with live syntax highlight overlay
Disasm panel	`#expert-disasm-btn`	Pure 6502 disassembly: each instruction shows address, hex bytes, and numeric operands; macros fully expanded

Toolbar buttons

Button	ID	Function
Format	`#expert-format-btn`	Auto-format source (labels to col 0, 4-space indent, 1-space mnemonic/operand)
Load .asm	`#expert-load-asm-btn`	Open a `.asm` file — the content is loaded into a **new tab** with the filename as the tab label. Each loaded file becomes an independent tab with its own program blocks and editor state.
Save .asm	`#expert-save-asm-btn`	Save editor content to a `.asm` file (file dialog on first save)
Build Info	`#expert-build-info-btn`	Open the Build Info dialog (origin, size, labels, errors)
HL	`#expert-hl-btn`	Toggle syntax highlighting (disable for very large files)
Palette	`#expert-palette-btn`	Show/hide the left mnemonic palette
Disasm	`#expert-disasm-btn`	Show/hide the disassembly panel (pure 6502, macros expanded)
Monitor	`#expert-monitor-btn`	Show/hide the monitor hex-dump panel

Error highlighting

Lines that fail to compile are highlighted in **red** (tinted background + left accent border) in real time, 350 ms after each keystroke. The first error message is also shown in the status bar. Fix the line and the highlight disappears automatically.

Syntax highlight

The editor uses a transparent `<div>` overlay (`expert-hl`) that mirrors the textarea content with coloured `<span>` elements. Highlight can be toggled off with the **HL** button for performance on very large programs.

Colour	Token
Yellow-green	Mnemonics (`LDA`, `STA`, `JMP`, …)
Blue	Directives (`.byte`, `.word`, `.fill`, `*=`, …)
Orange	Numbers (`$FF`, `%1010`, `255`)
Cyan	Labels (lines ending in `:`)
Teal	String literals
Dark green	Comments (`; …`)

Source formatter

Click the **Format** button (`#expert-format-btn`) to auto-format the current source:

- Label definitions are moved to column 0.
- Instructions are indented with 4 spaces.
- Mnemonics are uppercased.
- Exactly one space between mnemonic and operand (extra whitespace is normalised).
- If the source is already formatted, a `"Already formatted"` status is shown.

Project panel & tabs

Expert mode supports a **project panel** (`#expert-project-panel`) for multi-file `.proj` projects:

- A `.proj` file is a JSON manifest that lists source files and their metadata.
- Open a project with **Menu** → **File** → **Open project** or drag a `.proj` file onto the window.
- Each file in the project opens as a separate **tab** in the tab bar at the top of the editor.
- **Close Project** (`Menu → File → Close project` / `#menu-close-project`) closes the current project and all its file tabs at once. Prompts to save any unsaved changes before closing. The project panel resets to its empty state and `_expertProjectData` is cleared.
- Each file can be marked as the **startup file** (★ star icon). When a startup file is set, the **Run** button (PRG, D64, Ultimate) always assembles and runs that file's code — regardless of which tab is currently active. This works in both block mode and Expert mode.

Tab bar

The tab bar appears above the editor when there is more than one tab open.

Feature	Description
Dirty dot	A small accent-coloured dot on the tab name indicates unsaved changes

Feature	Description
Scroll arrows	Left/right scroll buttons appear when there are more tabs than fit the bar
Close (×)	Closes the tab; prompts to save if the tab is dirty
File extension	The full filename including extension (`.c64va` , `.json`) is shown

> **Tip:** Palette sync (`#expert-palette-sync-btn`) keeps the palette selection in sync with the mnemonic at the cursor. Disable it when you prefer not to have the palette jump around as you edit.

---

7. Addressing Modes

Each 6502 instruction supports one or more addressing modes. The mode selector appears on each block.

Mode	Label	Example	Description
implied	Implied	`NOP`	No operand; the instruction is self-contained
immediate	Immediate	`LDA #$FF`	Inline constant; the assembler adds `#` automatically
zeroPage	Zero page	`LDA $10`	Single byte address in page zero (0–255)
zeroPageX	Zero page,X	`LDA $10,X`	Zero page address + X register offset (result wraps in page 0)
zeroPageY	Zero page,Y	`LDX $FB,Y`	Zero page address + Y register offset
absolute	Absolute	`LDA $0400`	Full 16-bit memory address
absoluteX	Absolute,X	`LDA $0400,X`	16-bit address + X register offset
absoluteY	Absolute,Y	`LDA $0400,Y`	16-bit address + Y register offset
relative	Relative/Label	`BNE loop`	For branch instructions; enter a label name or target address
indirectX	Indirect,X	`LDA ($FB,X)`	Zero page indexed indirect (operand = zero page address, 1 byte)
indirectY	Indirect,Y	`LDA ($FB),Y`	Zero page indirect indexed (operand = zero page address, 1 byte)
indirect	Indirect	`JMP ($0100)`	Indirect; only usable with JMP

Label expressions as operands

Any operand field that accepts an address or immediate value also accepts a **constant name** (from a `CONST` block or a `LABEL`) directly. Additionally, you can use **label+offset** or **label−offset** expressions to reference an address relative to a named constant:

Syntax	Example	Description
`label`	`STA screen_ram,X`	Resolves to the label/constant value
`label+$hex`	`STA screen_ram+$0100,X`	Label address plus a hex offset
`label+decimal`	`STA screen_ram+256,X`	Label address plus a decimal offset

Syntax	Example	Description
`label-$hex`	`LDA table-$10`	Label address minus a hex offset
`#<label`	`LDA #<screen_ram`	Low byte of the label address
`#>label`	`LDA #>screen_ram`	High byte of the label address
`*`	`BNE *`	Current program counter (the instruction's own address); branches with `*` generate an infinite self-loop (offset `$FE`)

Example — clear two screen pages using a CONST:

```
; .CONST screen_ram = $0400
    LDX #$00
clear:
    STA screen_ram,X
    STA screen_ram+$0100,X
    DEX
    BNE clear
```

---

8. Standard 6502 Instructions

Data Movement

Mnemonic	Description	Modes
`LDA`	Load Accumulator	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`LDX`	Load X register	immediate, zeroPage, zeroPageY, absolute, absoluteY
`LDY`	Load Y register	immediate, zeroPage, absolute, absoluteX
`STA`	Store Accumulator	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`STX`	Store X register	zeroPage, zeroPageY, absolute
`STY`	Store Y register	zeroPage, absolute

Arithmetic

Mnemonic	Description	Notes
`ADC`	Add with carry	Set carry with `SEC` before use in most cases
`SBC`	Subtract with carry	Set carry with `SEC` before subtraction
`INC`	Increment memory	—

Mnemonic	Description	Notes
`DEC`	Decrement memory	—
`CMP`	Compare with A	Sets flags; does not modify A
`CPX`	Compare with X	—
`CPY`	Compare with Y	—

Logic

Mnemonic	Description
`AND`	Logical AND with Accumulator
`ORA`	Logical OR with Accumulator
`EOR`	Exclusive OR with Accumulator
`BIT`	Test bits in memory against A (sets N, V, Z flags)

Jumps & Branches

Mnemonic	Description
`JMP`	Unconditional jump (absolute or indirect)
`JSR`	Jump to subroutine (saves return address on stack)
`RTS`	Return from subroutine
`RTI`	Return from interrupt
`BNE`	Branch if Not Equal (Z=0)
`BEQ`	Branch if Equal (Z=1)
`BCC`	Branch if Carry Clear (C=0)
`BCS`	Branch if Carry Set (C=1)
`BMI`	Branch if Minus (N=1)
`BPL`	Branch if Plus (N=0)
`BVC`	Branch if Overflow Clear (V=0)
`BVS`	Branch if Overflow Set (V=1)

Register Operations

Mnemonic	Description
`TAX`	Transfer A → X
`TAY`	Transfer A → Y
`TXA`	Transfer X → A

Mnemonic	Description
`TYA`	Transfer Y → A
`TSX`	Transfer Stack pointer → X
`TXS`	Transfer X → Stack pointer
`INX`	Increment X
`DEX`	Decrement X
`INY`	Increment Y
`DEY`	Decrement Y

Shift & Rotate

Mnemonic	Description
`ASL`	Arithmetic Shift Left
`LSR`	Logical Shift Right
`ROL`	Rotate Left through Carry
`ROR`	Rotate Right through Carry

Stack

Mnemonic	Description
`PHA`	Push Accumulator onto stack
`PHP`	Push Processor status onto stack
`PLA`	Pull Accumulator from stack
`PLP`	Pull Processor status from stack

System / Flags

Mnemonic	Description
`CLC`	Clear Carry flag
`CLD`	Clear Decimal mode
`CLI`	Clear Interrupt disable
`CLV`	Clear Overflow flag
`SEC`	Set Carry flag
`SED`	Set Decimal mode
`SEI`	Set Interrupt disable
`NOP`	No operation

Mnemonic	Description
`BRK`	Force break / software interrupt

Illegal / Undocumented Instructions

These are supported for advanced use. Use with care — behavior may differ between chips.

`LAX`, `SAX`, `DCP`, `ISC`, `SLO`, `RLA`, `SRE`, `RRA`, `ANC`, `ALR`, `ARR`, `AXS`

---

9. Macro Blocks — Reference

Macro blocks let you do common tasks in one step — instead of writing 10–20 instructions by hand, you drop one block and the assembler generates the code for you. Think of them as built-in subroutines.

LABEL

Like a **line number in BASIC** — but with a name instead of a number. Jump targets for `JMP`, `JSR`, `BNE`, etc.

Field	Description
Label name	Identifier used in `JMP`, `JSR`, `BNE`, etc.

Generated ASM:

```
loop:  ; $0820
```

The current address is shown as a comment. Labels have **0 byte size**.

COMMENT

Like **REM in BASIC** — a note for yourself that the assembler ignores completely.

Generated ASM:

```
; Your comment text here
```

BYTE

Like **DATA in BASIC** — stores a list of raw byte values inline in the program.

Field	Description
Operand	Comma-separated byte values (e.g. `$01, $02, $FF` or `1, 2, 255`)

Generated ASM:

```
.byte $01, $02, $FF
```

Size: Number of bytes in the list.

WORD

Like **DATA in BASIC but for 16-bit numbers**. Each value is stored as two bytes (low byte first, then high byte — 6502 little-endian order).

Field	Description
Operand	Comma-separated 16-bit values (e.g. `$0400, $C000`)

Generated ASM:

```
.word $0400, $C000
```

Size: 2 bytes per word.

FILL

Like `FOR I=1 TO N : POKE addr+I, val : NEXT` — fills a block of memory with the same byte, but in a single block. Great for clearing areas or pre-filling tables.

Field	Description
Operand	`count,value` — e.g. `256,0` fills 256 bytes with zero

Generated ASM:

```
.fill 256, $00
```

Size: The count value in bytes.

ALIGN

Slides the current address forward to the next clean boundary by inserting zero-padding bytes. The C64 requires sprite data to start on a 64-byte boundary — `ALIGN 64` handles that automatically.

Field	Description
Boundary	Alignment value — e.g. `64` (sprite boundary), `256` (page), `$2000` (bitmap)

Generated ASM:

```
; ALIGN 64 → $0840 (12 bytes padding)
```

Size: Dynamic — depends on the current program counter position.

> **Tip:** Use `ALIGN 64` before sprite data, `ALIGN 256` to ensure page-aligned tables.

TEXT

Like **PRINT AT** — writes text directly to the C64 screen at a given column and row, without using the KERNAL. It generates one LDA/STA pair per character, targeting screen RAM at `$0400`.

Field	Description
Text	The string to display
X	Column (0–39)
Y	Row (0–24)
Label (optional)	Assigns a label pointing to the computed screen address

Case auto-detection: If the text contains any letter, the macro uses lowercase charset screen codes (`a` – `z` → 1–26). This works correctly when the C64 charset is set to lowercase mode (`LDA #$17 : STA $D018`). Numbers and symbols always use standard screen codes.

Generated ASM:

```
LDA #$08      ; 'H' (screen code, upper-case in standard charset)
STA $0400
LDA #$05      ; 'E' (screen code)
STA $0401
...
```

Characters are encoded as **screen codes** (not PETSCII). **Size:** `text.length × 5` bytes (LDA + STA per character).

STRING

Like **POKEing a string** into any memory address at runtime. Generates LDA/STA pairs that copy each character's screen code to consecutive addresses. Same case auto-detection as TEXT.

Field	Description
Text	The string to write
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address
Shift	Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video)

Expert syntax:

```
.string $C000, "HELLO"              ; hex address
.string my_buf, "HELLO"             ; label address (resolved at assembly time)
.string $C000, "HELLO" :my_string   ; with macroLabel
```

Generated ASM:

```
LDA #$08      ; 'H' screen code
STA $C000
LDA #$05      ; 'E' screen code
STA $C001
...
```

Characters are encoded as **screen codes** (not PETSCII). The optional **Shift** value is added to every byte, e.g. `$80` for reverse video. **Size:** `text.length × 5` bytes (one LDA + one STA per character).

DATA

Like a **POKE loop** — writes a list of raw bytes to a memory address at runtime, one LDA/STA pair per byte.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address

Expert syntax:

```
.data $C000, $01, $02, $03          ; hex address
.data my_buf, $01, $02, $03         ; label address
.data $C000, $01, $02, $03 :mydata  ; with macroLabel
```

Generated ASM:

```
    LDA #$01
    STA $C000
    LDA #$02
    STA $C001
    ...
```

Size: `byte_count × 5` bytes (one LDA + one STA per byte).

RAWBYTES

Like **DATA that loads directly into memory** — no runtime code at all. The bytes are present from the moment the PRG loads, before your code even starts. Use this for sprite data, level maps, lookup tables, anything that just needs to be at a specific address.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address

Expert syntax:

```
.rawbytes $C000, $00, $00, $00      ; hex address
.rawbytes sprite_data, $00, $00     ; label address
.rawbytes $0C50, $00, $00 :nev      ; with macroLabel — other code can use LDA nev,X
```

Size in code: 0 bytes. The data is placed at the given address in the output.

> **DATA vs RAWBYTES:** DATA generates LDA/STA code that copies bytes at runtime (slower, but works if the data needs to be dynamic). RAWBYTES just places the bytes directly — no code, instant, zero cost.

RAWTEXT

Like RAWBYTES but for text — encodes the string as screen codes and places the bytes at a fixed address with **no runtime code**. The text is ready in memory the instant the PRG loads. Same case auto-detection as TEXT and STRING.

Field	Description
Text	String to encode
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address
Shift	Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video)

Expert syntax:

```
.rawtext $C000, "HELLO"             ; hex address
.rawtext screen_pos, "HELLO"        ; label address
.rawtext $0400, "HELLO" :my_text    ; with macroLabel
```

Generated ASM:

```
; .rawtext "HELLO" -> $C000
; $C000
    .byte $08, $05, $0C, $0C, $0F   ; H E L L O (screen codes)
```

Size in code: 0 bytes. The data is placed at the given address in the output.

> **STRING vs RAWTEXT:** STRING generates LDA/STA code that copies the text at runtime. RAWTEXT bakes the bytes into the

> PRG at load time — no code, no waiting.

PETSCII

Like **RAWBYTES but for KERNAL output** — encodes the string as PETSCII bytes (compatible with CHROUT at `$FFD2`) and places them at a fixed address with no runtime code. Use this when you want to print characters via `JSR $FFD2` in a loop.

Field	Description
Text	String to encode as PETSCII bytes
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address

Expert syntax:

```
.petscii $C000, "HELLO"            ; hex address
.petscii msg_buf, "HELLO"          ; label address
.petscii $C000, "HELLO", null      ; with null terminator
.petscii $C000, "HELLO" :my_msg    ; with macroLabel
.petscii $C000, "HELLO", null :msg  ; combined
```

Generated ASM:

```
; .petscii "HELLO" -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; H E L L O
```

Size in code: 0 bytes. The data is placed at the target address as a deferred data section (like RAWBYTES).

Null terminator: Check the "*Append `$00` (null terminator)*" checkbox in the PETSCII block to automatically add a `$00` byte after the text. This is ideal for null-terminated string loops:

```
    LDX #$00
for0:
    LDA msg,X
    BEQ done        ; $00 stops the loop
    JSR $FFD2
    INX
    CPX #$20
```

```
    BNE for0
done:
    RTS
```

In expert mode, add `, null` after the text: `.petscii $C000, "HELLO", null`

Encoding rules:

Input	Byte value
Printable ASCII (space, letters, digits, punctuation)	Standard ASCII code (32–126)
Newline	`$0D` (RETURN)
Everything else	`$20` (space)

> **Tip:** Use PETSCII for data that will be output via CHROUT (`$FFD2`). For screen-code strings (different mapping), use the STRING macro instead.

INCBIN

Like **BLOAD in BASIC** — picks up an external binary file (`.bin`, `.prg`, `.sid`, `.raw`) and embeds it directly into the assembled PRG at the address you specify.

Field	Description
File	Browse to select a `.bin`, `.prg`, `.sid`, or `.raw` file
Address	Target load address (e.g. `$C000`)

Generated ASM comment:

```
    ; INCBIN "music.bin" @ $C000 (2048 bytes)
    .byte $01, $02, ...
```

Size in code: 0 bytes (deferred data section). The binary is embedded at the given address.

SID

Like **BLOAD for music** — loads a `.sid` file into your PRG and automatically reads its Init and Play addresses from the header. Call Init once at startup, then call Play from your IRQ handler every frame.

Field	Description
File	Browse to select a `.sid` file

Field	Description
Custom address (optional)	Override the SID's native load address (e.g. `$1000`). Leave empty to use the address from the SID header.

The block displays:

- **Title / Author** from the SID header
- **Load address** — where the data is placed in memory (effective address after any override)
- **Init address** — call this with JSR to initialize the music (adjusted for relocation if a custom address is used)
- **Play address** — call this with JSR on every frame in an IRQ handler (adjusted for relocation)
- A **(relocated)** badge appears when a custom address shifts the data from its original position

Generated ASM comment:

```
; SID "Ikari_Warriors.sid" @ $1000  Init:$1000  Play:$1006  (4096 bytes)
```

Size in code: 0 bytes inline. The SID binary is placed at the specified address as a deferred chunk in the PRG.

> **Important:** Most SID files contain hardcoded internal absolute addresses. They can only be relocated if the entire binary is shifted by the same offset. If a SID has internal jumps to `$10xx`, it must remain at `$1000` — moving it to a different address will break those internal references.

> **Typical usage:** Place an ORG block before the SID block to set its address. Call Init once at startup, then call Play every frame from a raster IRQ handler.

INCLUDE

Like **MERGE in BASIC** — pulls in another Visual Assembler project file and expands its blocks inline at this position. Perfect for reusable subroutine libraries. The included blocks are read-only in the current project.

Field	Description
File	Browse to select a `.json` Visual Assembler project
Load address (optional)	If set (hex, e.g. `C000`), the included blocks are placed at that address — a synthetic `ORG` is injected before them, overriding any ORG block inside the included file. Leave empty to let the included file's own ORG blocks control placement.

Generated ASM (no address override):

```
; .include "library.json" — 12 block(s)
... (expanded blocks follow)
```

Generated ASM (with load address `C000`):

```
; .include "library.json" @ $C000 — 12 block(s)
*=$C000
... (expanded blocks follow)
```

> **Tip:** Use INCLUDE to build reusable subroutine libraries that you can share across projects. Set a load address when the library has no ORG of its own, or when you want to override its default placement.

TABLE

Like **DIM at a specific address** — names a lookup table and sets where it lives in memory. Place BYTE, WORD, or FILL blocks after it to define the table's contents.

Field	Description
Name	Label identifier for the table (e.g. `color_table`)
Address	Fixed address where the table starts (e.g. `$C000`)

Generated ASM:

```
color_table:
```

The program counter jumps to the specified address. Place BYTE/WORD/FILL blocks after TABLE to fill the content.

Size: 0 bytes.

ORG

Sets where in memory the program (or a section of it) is placed — like choosing a start address before typing in machine code. Every program needs at least one ORG. The standard C64 BASIC-loadable start is `$0801`.

Field	Description
Address	The new origin address (e.g. `0801` in HEX, or `2049` in DEC)
HEX / DEC	Toggle the address input between hexadecimal and decimal display

Generated ASM:

```
* = $C000
```

Size: 0 bytes. The ORG block itself generates no machine code.

Each ORG block starts a new section. Blocks that follow are assembled starting at that address. When you export the PRG, all sections are merged into one file — gaps between sections are filled with zeros.

Example — code at `$0801`, data table at `$C000`:

```
* = $0801
    LDX #$00
loop:
    LDA $C000,X
    STA $D800,X
    INX
    BNE loop
    RTS

* = $C000
    .byte $01, $02, $03, ...
```

> **Tip:** Every program must start with an ORG block. The typical starting address for a C64 BASIC-loadable program is `$0801` (2049 decimal). When **BASIC SYS stub** is enabled, the assembler adds a short BASIC line at `$0801` and your code starts at `$080D`.

LOOP / NEXT

Like `FOR X=N TO 1 STEP -1 : ... : NEXT X` in BASIC — counts down from N to 1 using the X or Y register. Drop a LOOP block, put your instructions between it and NEXT, and it loops the right number of times automatically.

LOOP

Field	Description
Register	`X` or `Y` — the counter register
Count	Loop iteration count (hex or decimal, e.g. `0A` = 10)
Label	Auto-generated loop label (e.g. `loop0`)

Generated ASM:

```
    LDX #$0A
loop0:
```

Size: 2 bytes (LD_ opcode + immediate operand).

NEXT

Field	Description
Register	Automatically matched to the LOOP register
Label	Automatically linked to the LOOP label

Generated ASM:

```
    DEX
    BNE loop0
```

Size: 3 bytes (DEX + BNE + branch offset).

Example — clear 10 screen cells:

```
    LDX #$0A
loop0:
    LDA #$20        ; space character
    STA $0400,X
    DEX
    BNE loop0
```

FOR / ENDF

Like **`FOR X=0 TO N-1 : ... : NEXT X`** in BASIC — counts *up* from 0. Ideal when you need a forward index, e.g. stepping through a string or an array.

FOR

Field	Description
Register	`X` or `Y` — the counter register

Field	Description
Count	Loop limit (hex or decimal, e.g. `$12` = 18). X/Y runs from 0 up to limit-1.
Label	Auto-generated loop label (e.g. `for0`)

Generated ASM:

```
    LDX #$00
for0:
```

Size: 2 bytes (LD_ opcode + `#$00`).

ENDF

Field	Description
Register	Automatically matched to the FOR register
Label	Automatically linked to the FOR label
Count	Automatically copied from the paired FOR

Generated ASM:

```
    INX
    CPX #$12
    BNE for0
```

Size: 5 bytes (IN_ + CP_ #imm + BNE offset).

Example — print a null-terminated string:

```
    LDX #$00
for0:
    LDA msg,X       ; msg = PETSCII string at fixed address
    BEQ done        ; null terminator → exit
    JSR $FFD2       ; CHROUT
    INX
    CPX #$12        ; 18 characters max
    BNE for0
done:
    RTS
```

> **LOOP vs FOR:** LOOP counts down (N→1) — good for delays, fills, pixel loops. FOR counts up (0→N) — good for string/array access. Both can use X or Y.

PUSH / PULL

Like **saving variables before a GOSUB and restoring them after** — but uses the 6502 hardware stack. If a subroutine uses A, X, or Y, wrap it with PUSH and PULL so the calling code's registers are preserved.

PUSH

Pushes one or more registers onto the stack. The order is always A → X → Y (innermost first).

Field	Description
Registers	Any combination: `A`, `X`, `Y`, `AX`, `AY`, `XY`, `AXY`

Generated ASM (example: `AX`):

```
PHA
TXA
PHA
```

Size: 1 byte for A (`PHA`), 2 bytes for X or Y (transfer + push).

PULL

Restores registers from the stack in **reverse order** (Y → X → A).

Field	Description
Registers	Same as PUSH — must match the corresponding PUSH block

Generated ASM (example: `AX`):

```
PLA
TAX
PLA
```

> **Rule:** PUSH and PULL must always use the **same register set**. `PUSH AX` → `PULL AX` (internally restores in reverse: X first, then A).

MACRO / ENDM / INVOKE

Like **a named GOSUB with parameters** — define a reusable chunk of code once (MACRO...ENDM), then call it anywhere with INVOKE. Pass different argument values each time instead of copy-pasting blocks.

MACRO (definition start)

Field	Description
Name	Identifier for the macro (e.g. `setColor`)
Params	Optional comma-separated parameter names (e.g. `color` or `color, count`)

Marks the start of a macro definition. Blocks between MACRO and ENDM are the macro's body — they **don't generate any code** where the definition sits. Use `{paramName}` as a placeholder for arguments.

Generated ASM:

```
; .MACRO setColor (color)
    ... (body blocks)
; .ENDM
```

Expert mode syntax:

```
.macro setColor color
    LDA {color}
    STA $D020
.endm
```

ENDM (definition end)

Closes the current macro definition. No fields.

INVOKE

Calls a defined macro at this position and substitutes the supplied argument values for `{paramName}` placeholders in the body.

Field	Description
Macro name	Select from the dropdown of defined macros
Arguments	Comma-separated argument values matching the macro's parameter list (e.g. `#$07`)

Generated ASM:

```
; .invoke setColor(#$07)
    LDA #$07
    STA $D020
```

Expert mode syntax:

```
; single argument:
.invoke setColor(#$07)

; multiple arguments:
.invoke drawPixel($10, $20)

; no arguments:
.invoke clearScreen

; text / string arguments (quoted):
.invoke printText("Hello, World!")

; .call alias (synonym for .invoke):
.call setColor(#$07)
```

The macro body is expanded inline with `{paramName}` replaced by the actual arguments. The space-separated form (`.invoke setColor #$07`) is also accepted.

Argument types:

- **Numeric**: `#$07`, `$10`, `255` — hex or decimal values
- **Text strings**: `"Hello, World!"` — quoted strings; commas inside quotes are treated as part of the text, not as argument separators
- **Mixed**: `#$07, "hello", $20` — any combination

> **Tip:** Define macros at the top (or bottom) of your program, then INVOKE them wherever needed. Macros can be invoked multiple times with different arguments.

REGION / ENDREGION

Purely visual grouping — **zero bytes**, zero effect on the assembled code. Like folding a section of a BASIC program into a named block so you can collapse it and focus on something else.

Field	Description
Region name	Free-text label for the section (e.g. `init`, `game_loop`, `sprite_setup`)

Controls on the REGION block header (always visible):

- ▸ **/ ▾ toggle** — collapses or expands the entire region. When collapsed, all blocks between REGION and ENDREGION are hidden.
- ↕ **Expand all** — un-collapses every individually collapsed block inside the region and expands the region itself if needed.
- ⊖ **Select in ASM** — highlights the entire region's code range in the ASM view (from `; ===[ name ]===` to `; ===[/name]===`) and scrolls to it. Switches to the ASM tab automatically if it is not currently visible.
- ⧉ **Copy region** — copies the REGION block, all child blocks, and the matching ENDREGION into a clipboard. A ✓ flash confirms the copy.
- ⎘ **Paste region** — inserts the copied region as a new region immediately after the current region's ENDREGION and scrolls to it. The button is dimmed until a region has been copied.

Generated ASM:

```
; region init
    SEI
    LDA #$00
    STA $D020
; endregion init
```

Size: 0 bytes for both REGION and ENDREGION.

Example workflow:

1. Add a `REGION` block, set region name to `init`.
2. Add your initialization instructions below it.
3. Add an `ENDREGION` block to close the section.
4. Click ▸ on the REGION to collapse the whole section into one line while working on other parts of the program.

> **Note:** Regions can be **nested** inside each other. Each ENDREGION closes the nearest open REGION. No effect on the assembled output.

DEFINE / IF / ELSE / ENDIF

Like **a switch the assembler reads** — `DEFINE DEBUG` turns on a symbol, then any `IF DEBUG` block is included and its `ELSE` branch is skipped. Remove the DEFINE block and the IF block disappears from the output. No need to delete code for release builds.

DEFINE

Field	Description
Symbol	One or more comma-separated identifiers to activate (e.g. `DEBUG` or `DEBUG, PAL`)

Generated ASM:

```
; .DEFINE DEBUG
; .DEFINE DEBUG, PAL
```

A `DEFINE` block can activate multiple symbols at once (comma-separated). Place DEFINE blocks at the top of your program. Removing the block deactivates all its symbols instantly.

IF

Field	Description
Condition	Identifier to test (must match a `DEFINE` symbol to be active)

Generated ASM:

```
; .IF DEBUG
```

Blocks between `IF` and `ENDIF` (or `ELSE`) are included or skipped based on whether the condition symbol has a matching `DEFINE` in the program. Skipped blocks appear as `; [IF skipped] …` comments and generate **zero bytes**.

ELSE

No fields. Marks the alternative branch — assembled when the `IF` condition is *not* active.

Generated ASM:

```
; .ELSE
```

ENDIF

No fields. Closes the conditional block.

Generated ASM:

```
; .ENDIF
```

Size: 0 bytes for all four blocks. Only the content *between* them counts.

Example — debug border flash, release build skips it:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02        ; red border
    STA $D020
; .ELSE
    LDA #$00        ; black (release)
    STA $D020
; .ENDIF
```

Example — multiple symbols in one DEFINE block:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx        ; PAL timing constant
; .ELSE
    LDA #$xx        ; NTSC timing constant
; .ENDIF
```

Nested `IF` blocks are supported. If an outer block is skipped, inner blocks are skipped too.

CONST

Like **a named variable that never changes** — `SCREEN = $0400`. Use the name instead of typing raw addresses everywhere, making the code easier to read and change later.

Field	Description
Name	Identifier for the constant (e.g. `SCREEN`)
Value	Numeric value in the selected base (e.g. `0400` in HEX = address $0400)
Format	HEX or DEC — controls how the value is entered and displayed

Generated ASM:

```
; .CONST SCREEN = $0400
```

The constant name appears in the **label picker** dropdown on instruction blocks — just click it to insert.

Size: 0 bytes.

SPRITE_INIT

Sets up a VIC-II sprite in one block — instead of writing ~6 POKE statements in BASIC, just fill in three fields. Sets the sprite's data pointer, turns it on, and sets its colour.

Field	Description
Sprite #	Sprite number 0–7
Colour	Colour index 0–15 (C64 palette)
Data page	Sprite data address / 64 (e.g. `$21` if data is at `$0840`)

Generated ASM:

```
        LDA #$21
        STA $07F8       ; sprite pointer register ($07F8 + N)
        LDA $D015
        ORA #$01        ; set enable bit for sprite 0
        STA $D015
        LDA #$07
        STA $D027       ; sprite 0 colour register
```

Size: 18 bytes.

> **Sprite data page:** `data_address ÷ 64`. With the default BASIC SYS stub, `ALIGN 64` after `JMP main` places sprite data at `$0840` → page = `$21`.

SPRITE_POS

Like **`POKE 53248, x : POKE 53249, y`** in BASIC — sets a sprite's starting position. The coordinates are baked in at assemble time; for animation use `INC` / `DEC` on the sprite register directly.

Field	Description
Sprite #	Sprite number 0–7
X	Horizontal position 0–319
Y	Vertical position 0–255

Generated ASM (example: sprite 0, X=152, Y=100):

```
    LDA #$98        ; X low byte
    STA $D000       ; sprite 0 X register
    LDA $D010
    AND #$FE        ; clear X MSB for sprite 0 (X ≤ 255)
    STA $D010
    LDA #$64        ; Y = 100
    STA $D001       ; sprite 0 Y register
```

For X > 255 the macro sets the corresponding bit in `$D010` instead of clearing it.

Size: 18 bytes.

> **Note:** `SPRITE_POS` bakes the X/Y into the code (`LDA #$xx`). To animate a sprite at runtime use `INC $D000` / `DEC $D000` — see the `sprite-macro-demo` sample.

WAIT_RASTER

Waits for the VIC-II electron beam to reach a specific scan line — like syncing to a TV frame. Put this at the top of your game loop to prevent sprite tearing. No JSR, no label needed.

Field	Description
Raster line	Target raster line in hex (e.g. `FF` = line 255)

Generated ASM:

```
wait:
    LDA $D012       ; current raster line
    CMP #$FF        ; target line
    BNE wait        ; loop back (-7 bytes)
```

Size: 7 bytes (the `BNE` offset `$F9` = −7 always points back to the `LDA`).

> **Tip:** Place `WAIT_RASTER` at the top of your game loop to synchronise with the display and prevent sprite tearing.

JOYSTICK

Like reading **`PEEK($DC00)`** and then POKEing the sprite position — but in one block. Reads one CIA joystick port and adjusts a sprite's X/Y registers accordingly. Entirely inline, no JSR needed.

Field	Description
Port	`1` = port 1 (`$DC01`) or `2` = port 2 (`$DC00`)
Sprite #	Sprite number 0–7 (controls which X/Y register pair is updated)

Generated ASM (port 2, sprite 0):

```
    LDA $DC00       ; read CIA port 2
    LSR             ; bit 0 → carry (Up)
    BCS skip_up     ; carry set = NOT pressed
    DEC $D001       ; Y-1 (move up)
skip_up:
    LSR             ; bit 1 → carry (Down)
    BCS skip_down
    INC $D001       ; Y+1 (move down)
skip_down:
    LSR             ; bit 2 → carry (Left)
    BCS skip_left
    DEC $D000       ; X-1 (move left)
skip_left:
    LSR             ; bit 3 → carry (Right)
    BCS skip_right
    INC $D000       ; X+1 (move right)
skip_right:
```

Joystick bit map (active-LOW — bit = 0 means pressed):

Bit	Direction	CIA register
0	Up	$DC00 (port 2) / $DC01 (port 1)
1	Down	
2	Left	
3	Right	
4	Fire	(not handled by this macro)

Size: 27 bytes. The `BCS` offset is always `+3` (skips the following 3-byte `DEC` / `INC abs` instruction).

> **Typical usage:** Place inside a `gameloop` label with `WAIT_RASTER` first:
>
> ```
> gameloop:
>     WAIT_RASTER ($FF)
>     JOYSTICK (port=2, sprite=0)
>     JMP gameloop
> ```

MOUSE

Reads a Commodore 1351 proportional mouse and moves a sprite. Entirely **inline** — no JSR or label needed. The macro selects the CIA port, waits for the SID paddle inputs to settle, then decodes the delta movement using the standard 1351 driver pattern and applies it to the sprite registers.

Field	Description
Port	`1` = CIA `$DC00` bits `7:6` = `%01`; `2` = `%10`
Sprite #	Sprite number 0–7
ZP byte X	Zero-page address (hex) to hold the previous POTX sample (e.g. `FD`)
ZP byte Y	Zero-page address (hex) to hold the previous POTY sample (e.g. `FE`)

Generated ASM shape (port 1, sprite 0, ZP `$FD` / `$FE`):

```
    ; CIA port select + settle
    LDA $DC00
    AND #$3F
    ORA #$40
    STA $DC00
    LDX #$67
wait:
    DEX
    BNE wait

    ; X axis — standard 1351-style 7-bit delta decode
    LDA $D419
    TAY
    SEC
    SBC $FD
    AND #$7F
    LDX #$00
    CMP #$40
    BCS xneg
    LSR A
    BEQ xdone
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
```

```
    EOR #$01
    STA $D010
    JMP xdone
xneg:
    ORA #$C0
    CMP #$FF
    BEQ xdone
    SEC
    ROR
    DEX
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
xdone:

    ; Y axis — same decode, then inverted before apply
    LDA $D41A
    TAX
    SEC
    SBC $FE
    AND #$7F
    CMP #$40
    BCS yneg
    LSR A
    BEQ ydone
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
    JMP ydone
yneg:
    ORA #$C0
    CMP #$FF
    BEQ ydone
    SEC
    ROR
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
ydone:
```

Size: 142 bytes.

Expert mode syntax:

```
.mouse port, spriteNum, zpX, zpY
; example:
.mouse 2, 0, FD, FE
```

> **Important:** Before the first call, initialise the zero-page bytes with the current POTX/POTY values to avoid a jump on the first frame:
>
> ```
>     ; port 1: LDA $DC00 : AND #$3F : ORA #$40 : STA $DC00
>     ; port 2: LDA $DC00 : AND #$3F : ORA #$80 : STA $DC00
>     LDA $D419 : LSR A : AND #$3F : STA $FD
>     LDA $D41A : LSR A : AND #$3F : STA $FE
> ```

> **Tip:** Poll the mouse once per frame — place `WAIT_RASTER` in the game loop before `MOUSE`.

SPRITE_COL

Like **`PEEK($D01E)`** in BASIC — checks the VIC-II hardware collision registers and tells you whether a sprite hit another sprite or the background. Entirely inline, no JSR needed.

Field	Description
Sprite #	Sprite number 0–7 (which sprite's bit to check)
Collision type	`Sprite-Sprite ($D01E)` — collision with another sprite; `Sprite-Background ($D01F)` — collision with background graphics

Generated ASM (sprite 0, sprite–sprite):

```
    LDA $D01E       ; read sprite-sprite collision register (clears it!)
    AND #$01        ; isolate bit 0 (sprite 0)
                    ; A ≠ 0 → collision occurred
```

Size: 5 bytes.

> **Important:** Reading `$D01E` / `$D01F` **clears the register**. Read it once per frame and act on the result immediately with `BEQ` / `BNE`.

Typical usage:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    SPRITE_COL (sprite=0, type=sprite-sprite)
    BEQ no_hit          ; A = 0 → no collision
    LDA #$02
    STA $D020           ; red border = hit!
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; light blue border = clear
    JMP gameloop
```

> **See also:** `collision-demo` sample — green ball (sprite #0) vs. red cross (sprite #1).

LOADFILE

Like **`LOAD "file",8`** in BASIC — loads a file from a D64 disk at runtime using the KERNAL LOAD routine. Use this to load data, music, or extra code from disk while your program is running.

Field	Description
Filename	File name on the disk (max 16 chars, auto-uppercase; characters `,`, `"`, `/`, `\`, `:`, `*`, `?`, `<`, `>`, `|` are filtered)
Device	Device number 8–30 (default `8`)
Override address (optional)	Hex load address (e.g. `C000`). If set, the file is loaded to this address (`sec=0`, ignoring the PRG header). Leave empty to use the file's own 2-byte PRG header (`sec=1`).
Error label (optional)	If set, a `BCS` instruction is generated after JSR LOAD. If the KERNAL returns with carry set (error), execution jumps to this label.

Generated code structure:

```
    JMP skip_filename      ; jump over the inline filename
    .byte "DEMO-COLORS"    ; filename bytes (PETSCII, 11 chars)
skip_filename:
    LDA #11                ; filename length
    LDX #<fname            ; pointer lo
    LDY #>fname            ; pointer hi
```

```
        JSR $FFBD               ; SETNAM
        LDA #1                  ; logical file #1
        LDX #8                  ; device 8
        LDY #0                  ; sec=0 (override addr) or #1 (file's own addr)
        JSR $FFBA               ; SETLFS
        LDA #0                  ; LOAD command (not VERIFY)
        JSR $FFD5               ; LOAD
        BCS fail                ; (only if error label set)
```

Size: `3 + filename_length + 9 (SETNAM) + 9 (SETLFS) + (4 if override) + 5 (LOAD) + (2 if error label)` bytes. Minimum 27 bytes.

> **Important:** The filename is stored inline in the machine code right after a `JMP skip_filename`. The file name on the disk must be uppercase PETSCII — which matches plain ASCII uppercase letters (`A` – `Z`). The macro enforces this automatically.

> **Always use an error label** for production programs — if the file isn't found, KERNAL sets the carry flag and execution falls through to whatever is next.

> **See also:** `loadfile-demo` sample — demonstrates loading `DEMO-COLORS.PRG` from a D64 with a BCS error branch and a visual error screen.

EXODECRUNCH

In-program **Exomizer decompression**. Use this macro right after a `LOADFILE` that loaded an Exomizer `mem`-mode compressed stream — EXODECRUNCH unpacks it backward into the address embedded in the stream.

Field	Description
Depacker address	Where the depacker code lives in memory (default `B000`). Must be a 16-bit hex address.

Generated code (19 bytes):

```
        LDA $AE           ; KERNAL load-end lo (set by previous LOAD)
        STA $04           ; depacker src-end pointer lo
        LDA $AF           ; KERNAL load-end hi
        STA $05           ; depacker src-end pointer hi
        LDA #$36          ; BASIC ROM off ($A000-$BFFF becomes RAM)
        STA $01
        JSR depacker      ; depacker reads back-to-front via $04/$05
        LDA #$37          ; restore default mapping (BASIC + KERNAL + I/O)
        STA $01
```

How it works:

1. KERNAL `LOAD` ($FFD5) updates ZP `$AE/$AF` to point one past the last loaded byte. EXODECRUNCH copies this to ZP `$04/$05`, which is the official Exomizer backward source-end convention.
2. The depacker is typically placed at `$B000` (within the BASIC ROM mapped region). The macro toggles `$01 = $36` so the CPU sees RAM there during the JSR, then restores `$01 = $37` afterward.
3. The decompression target address is **encoded into the compressed stream itself** when you compress with `exomizer mem -l <load> file,<target>` — the depacker reads it from the stream's first bytes.

Depacker binary: the pre-built backward depacker is `samples/exo-decrunch.bin` (477 bytes, ORG $B000). It's a Kick Assembler wrap of the official `exodecrunch.asm` with `INC $D020` added to every read for a visible border-flash effect during decompression. Place it into your program with an `INCBIN` block at the depacker address.

Safety offset compensation: Exomizer's default mem mode applies a 2-byte safety offset — data lands 2 bytes earlier than the requested target. The Run via D64 dialog **automatically adds 2 to the Dst field** before calling exomizer, so the visible behaviour matches the address you typed.

> **See also:** the `exo-multicolor-demo` sample — full end-to-end example: LOADFILE a compressed multicolor bitmap to $C000, EXODECRUNCH unpacks it to $2000, then copy screen → $0400 and color → $D800, and switch VIC-II to multicolor bitmap mode.

> **Integration test:** `cargo test --test exomizer_integration` (in `src-tauri/`) verifies the full compression + decompression roundtrip on a 6502 emulator with the real depacker binary. Pass criteria: 10000 bytes byte-equal to the source `multi-color.bin`.

REU_CHECK

Detects whether a Commodore RAM Expansion Unit (REU) is plugged in — like checking `PEEK($D010)` to see if hardware is present. Tests by writing and reading back two patterns to REU register `$DF04`.

Generated code (34 bytes):

```
LDA #$55
STA $DF04
LDA $DF04
CMP #$55
BNE fail
LDA #$AA
STA $DF04
LDA $DF04
CMP #$AA
BNE fail
LDA #$00
CMP #$FF   ; Z=0 => REU present
BNE done
fail:
LDA #$FF
CMP #$FF   ; Z=1 => no REU
done:
```

> The macro normalizes the result so the following branch stays simple: `BNE` means REU present, `BEQ` means REU missing.

Result in flags:

- **Z = 0** (result ≠ 0) → REU present → use `BNE`
- **Z = 1** (result = 0) → no REU → use `BEQ`

No configurable fields — the macro generates the same code every time.

Typical usage:

```
REU_CHECK
BEQ no_reu        ; skip if REU not present
; ... REU code here ...
no_reu:
```

REU_STASH / REU_FETCH / REU_SWAP

DMA block transfer between C64 RAM and REU expansion memory — like a very fast POKE loop, but the CPU doesn't do any work (the REU chip copies the data while the CPU is halted). A `$1000`-byte transfer is effectively instant.

Macro	Direction	`$DF01` command
`REU_STASH`	C64 RAM → REU	`$90`
`REU_FETCH`	REU → C64 RAM	`$91`
`REU_SWAP`	C64 RAM ↔ REU	`$92`

Fields:

Field	Description	Example
C64 address	Source/dest in C64 RAM (hex)	`C000`
REU address	Source/dest in REU (hex, 16-bit)	`0000`
REU bank	REU memory bank (0–7)	`0`
Length	Number of bytes to transfer (hex, 16-bit)	`1000`

Generated code (40 bytes):

```
LDA #c64lo    STA $DF02    ; C64 address LO
LDA #c64hi    STA $DF03    ; C64 address HI
```

```
LDA #reuLo    STA $DF04    ; REU address LO
LDA #reuHi    STA $DF05    ; REU address HI
LDA #bank     STA $DF06    ; REU bank
LDA #lenLo    STA $DF07    ; length LO
LDA #lenHi    STA $DF08    ; length HI
LDA #$00      STA $DF09    ; address control (fixed)
LDA #$00      STA $DF0A    ; interrupt mask (fixed)
LDA #cmd      STA $DF01    ; execute DMA ($90/$91/$92 = stash/fetch/swap, immediate)
```

> **Note:** Commands use `$90/$91/$92` (bit 4 set = immediate DMA mode). Writing to `$DF01` starts the transfer; the CPU resumes when it finishes.

TURBO_SET

Sets the **Ultimate-64 (U64) CPU speed** via register `$D031`. No effect on a real C64 or other emulators.

Fields:

Field	Description	Range
Speed	CPU speed index	0 = 1 MHz … 7 ≈ 10 MHz … 15 ≈ 48 MHz
Badline	Badline emulation	Enabled (C64 compatible) / Disabled (turbo)

The speed byte is calculated as: `(speedIndex & 0x0F) | (badline_disabled ? 0x80 : 0x00)`.

Generated code (5 bytes):

```
A9 xx   LDA #speed_byte
8D 31 D0   STA $D031
```

Expert mode syntax:

```
.turbo_set 7,0    ; speed=7 (~10 MHz), badline enabled
.turbo_set 15,1   ; speed=15 (~48 MHz), badline disabled
```

> **Note:** This macro affects U64 hardware only. On a real C64 or other emulators this writes to `$D031` which may affect the CIA or be ignored.

SUPERCPU_DETECT

Checks whether a **CMD SuperCPU** accelerator is installed — like `PEEK($D0B8)` to see if it returns something other than `$FF`.

Generated code (5 bytes):

```
AD B8 D0   LDA $D0B8
C9 FF      CMP #$FF
```

Result in flags:

- **Z = 0** → SuperCPU present → use `BNE`
- **Z = 1** → SuperCPU not found → use `BEQ`

No configurable fields.

Typical usage:

```
SUPERCPU_DETECT
BEQ no_scpu       ; skip if SuperCPU not present
; ... SuperCPU turbo code here ...
no_scpu:
```

TURBO_ENABLE

Turns **CMD SuperCPU turbo mode** on or off. Call `SUPERCPU_DETECT` first and skip this if the SuperCPU isn't present.

Mode	Register	Effect
Enable	`$D07A`	Engage turbo (up to 20 MHz with SuperCPU)
Disable	`$D07B`	Return to 1 MHz compatibility mode

Generated code (5 bytes):

```
A9 00         LDA #$00
8D 7A D0      STA $D07A    ; (or $D07B for disable)
```

Expert mode syntax:

```
.turbo_enable on
.turbo_enable off
```

> **Note:** Call `SUPERCPU_DETECT` first and branch around this macro if the SuperCPU is not present.

---

10. Debugger Integration

The app supports **RetroDebugger** as the external C64 debugger. It receives breakpoints, symbols, and autostart flags generated from the assembled program.

RetroDebugger

RetroDebugger is a cross-platform Commodore 64 debugger with breakpoint support, memory inspection, and label-aware disassembly.

Setup: Open **Settings** → **Configure RetroDebugger executable** and point it to the `RetroDebugger` binary.

Launch: Click **Debug (RetroDebugger)** in the toolbar. The app will:

1. Assemble the program to a `.prg` file in a temporary directory.
2. Write a **breakpoints file** (`breakpoints.txt`) — one `break $ADDR` per flagged block.
3. Write a **symbols file** (`symbols.txt`) in Vice/RetroDebugger label format (`al C:addr .name`). All LABEL and CONST blocks are included.
4. Launch RetroDebugger with:

    ```
    RetroDebugger -prg <file.prg> -breakpoints <breakpoints.txt> -symbols <symbols.txt> [flags]
    ```

Breakpoint Blocks

Click the breakpoint icon (●) on any instruction block to toggle it as a breakpoint. Breakpointed blocks are highlighted in red. Their addresses are written to the breakpoints file on every debugger launch.

Debugger Flags (Options Tab)

Toggle	Flag	Effect
`-jmp` ON	`-jmp $ADDR`	Jump directly to the program start address after loading
`-unpause` ON	`-unpause`	Unpause the debugger immediately on load
`-wait` ON	`-wait <ms>`	Wait `<ms>` milliseconds before unpausing — 500 ms or 1000 ms

> **Tip:** For most programs, enable `-jmp` and `-unpause` for instant autostart. Use `-wait 500` or `-wait 1000` when your program sets up IRQs or SID music that needs time to initialize before the first raster.

---

11. Knowledge Base Links

Quick reference links available in the app under **Knowledge Base**:

Resource	URL
6502 Opcodes Reference	http://www.6502.org/tutorials/6502opcodes.html
C64 KERNAL Functions	https://sta.c64.org/cbm64krnfunc.html
C64 Memory Map	https://sta.c64.org/cbm64mem.html
C64 Color Codes	https://sta.c64.org/cbm64col.html
VIC-II Article	https://www.cebix.net/VIC-Article.txt
C64 Codebase	https://codebase.c64.org/
The Turbo Assembler	https://turbo.style64.org/
RetroDebugger	https://github.com/slajerek/RetroDebugger/

---

12. D64 Export & Run

Version 1.5.1 adds the ability to package your program (and additional data files) into a C64 D64 disk image and launch it in VICE — or export the disk image for use elsewhere.

Split Run button

The toolbar **Run** button has been replaced by a **split button**:

Part	Action
▶ **Run** (main)	Executes the currently selected run mode
▾ (arrow)	Opens the mode selector

Available run modes:

Mode	Description
Run as PRG	Assemble to a temporary `.prg` and launch VICE directly. Classic behaviour.
Run via D64	Assemble, build a `.d64` disk image (using c1541), add any configured extra files, then launch VICE from the disk. Use this whenever your program loads files at runtime (e.g. with the LOADFILE macro).
Run on hardware	Assemble to PRG and send it to a **1541 Ultimate / Ultimate 64** device over the local network. See Section 13.

The selected mode is saved between sessions.

Export to D64 dialog

Open via the **Save PRG ▾** dropdown → **Export to D64**. The dialog lets you:

1. Set the **disk name** (max 16 chars) and **program name** — these are the names that appear in the C64 disk directory.
2. **Add extra files** — click **+** to pick any binary file (`.prg`, `.bin`, `.sid`, etc.). For each extra:
   - **Name** — how it appears in the D64 directory (max 16 chars, auto-uppercase).
   - **Addr** (load address, optional) — if provided, a 2-byte PRG header is prepended. Leave empty to write raw bytes with no header.
   - **Dst** (decompression target, only with EXO) — where the depacker should land the data on the C64. When EXO is enabled, the extra is compressed with `exomizer mem -l <Addr> file,<Dst>` before being written to the D64. The +2 safety-offset compensation is applied automatically.
   - **EXO** — checkbox that turns on backward `mem`-mode crunching for this entry. The on-disk size is typically 5-20% of the original.
3. Click **Export** to generate the `.d64` file using VICE's `c1541` tool.

Pairing with EXODECRUNCH: when you ship a file with EXO=on, the program reading it should LOAD it to the **Addr** address (sec=1, file's own PRG header), then call the **EXODECRUNCH** macro to unpack it backward into **Dst**. See the `exo-multicolor-demo` sample for the complete pattern.

D64 metadata in projects

The disk name, program name, and extra file list are saved inside the project JSON (under the `d64` key). When you reload the project or a sample that includes D64 metadata, the extras are restored automatically — no need to re-add them each time.

The **loadfile-demo** sample comes pre-configured with `DEMO-COLORS.PRG` as an extra file. Select it, open **Run via D64**, and click **Run** to see the full load flow in action.

> **Requirement:** D64 export and Run via D64 both require VICE (`c1541`) to be configured in Hardware Settings.

---

13. Hardware Settings

Open via **Settings** → **Hardware Settings...** in the toolbar menu. All external hardware paths and network configuration live here.

VICE Emulator

Setting	Description
Select VICE	Browse to the `x64sc` (or `x64`) VICE executable
Status	Shows whether the executable path is valid and accessible

VICE is required for **Run as PRG**, **Run via D64**, and **Export to D64**.

Exomizer

Setting	Description
Select Exomizer	Browse to the `exomizer` executable
Border flash during decompression	When enabled, SFX-compressed PRGs use exomizer's built-in `-x1` fast border-flash effect; when disabled, `-n` is passed for silent decompression
Status	Shows whether the executable path is valid and accessible

Workflow:

1. Install the Exomizer binary:
   - **Windows:** download the pre-built `win32/exomizer.exe` from https://bitbucket.org/magli143/exomizer/wiki/Home or https://csdb.dk/release/?id=244342.
   - **macOS:** `brew install exomizer` (installs Magnus Lind's official 3.1.2 build).
2. Configure the path in **Hardware Settings** → **Exomizer section**.
3. Enable the **Run with Exomizer** checkbox in the **Settings menu**.
4. All **Run** actions (PRG, D64, hardware) and **Build** actions (Build PRG, Build D64) will now crunch the assembled program through `exomizer sfx sys` before launching or saving.

Exomizer works the same way on Windows and macOS — the CLI is invoked from the Tauri backend; nothing about the integration is platform-specific.

Two compression modes are used internally:

Mode	Used by	Calling convention
`sfx sys`	Build/Run with Exomizer toggle (main PRG path)	Self-extracting PRG with a built-in decruncher; the Border-flash setting controls `-x1` vs `-n`
`mem` (backward)	Run via D64 → per-file **EXO** checkbox	Compresses each extra file to a `mem`-mode stream; the program decompresses it at runtime via the **EXODECRUNCH** macro and a pre-built depacker (`samples/exo-decrunch.bin`)

> **Tip:** If the Exomizer path is not configured but the checkbox is enabled, a clear error toast is shown instead of launching. Disable the checkbox to run without compression.

> **Integration test:** `cargo test --test exomizer_integration` (in `src-tauri/`) verifies the full mem-mode compression + decompression roundtrip on a 6502 emulator.

Retro Debugger

Setting	Description
Select RetroDebugger	Browse to the `RetroDebugger` binary
Status	Shows whether the path is valid

See Section 9 for full debugger documentation.

C64 Ultimate / 1541 Ultimate

Run assembled PRGs directly on real hardware over the local network using the Ultimate REST API.

Setting	Description
Host (IP)	IP address of the device (e.g. `192.168.1.100`)
Password	Optional — if the device requires authentication
Test connection	Sends a test request to `/v3/runners/info`; shows OK or error

Workflow:

1. Connect the 1541 Ultimate / Ultimate 64 to your local network.
2. Enter its IP address (and password if set) in Hardware Settings.
3. Select **Run on hardware** from the split run menu.
4. Click ▶ **Run** — the PRG is compiled and sent to the device via HTTP POST to `/v3/runners/prg`. The device loads and runs it on the C64 immediately.

> **Tip:** No USB cable or driver needed — the REST API is built into the Ultimate firmware. Your computer and the device must be on the same local network.

---

14. Visual Editors (Toolkit)

The toolbar **Toolkit** menu groups the visual data editors that all share a common Files menu (`Files ▾`) for Load BIN / Save BIN / Export to blocks / Save to D64. Each editor produces raw `.bin` data that can be placed in a program with `INCBIN`, or directly added to a D64 disk via the **Save to D64** entry.

Hi-Res / Multicolor Editor

Pixel-level bitmap editor with both 320×200 hi-res and 160×200 multicolor modes. Open via Toolkit → Hi-Res Editor.

Feature	Description
Mode toggle	**Multicolor** checkbox switches between hi-res (mono per cell) and multicolor (4 colors per cell).
Tools	Pencil, eraser, line, rectangle, filled rectangle, oval, filled oval, flood fill.
Color palette	Foreground (ink) + paper (background) pickers. Multicolor mode tracks 3 per-cell extras automatically.
Undo / Redo	Per-stroke history, ctrl-Z / ctrl-Y.
Grid + Raster	Optional 8×8 grid and raster row overlay for cell alignment.
Import image	PNG/JPEG/GIF dropped into the canvas auto-quantizes to the 16-color C64 palette.
Export blocks	Appends BYTE/RAWBYTES blocks to the program with the encoded bitmap, screen, and color data.
Export `.bin`	Saves the native multicolor format (10000 bytes: 8000 bitmap + 1000 screen + 1000 color) ready for LOADFILE to $2000.

Sprite Editor

24×21 pixel sprite editor with multi-frame animation. Open via Toolkit → Sprite Editor.

Feature	Description
Frames	Add / remove / reorder frames; frame strip shown at the bottom.
Mode	Mono / multicolor toggle.
Tools	Pencil, fill, flip horizontal, flip vertical, shift left/right/up/down (with optional wrap).
Animation preview	Play / Stop with configurable speed.
Export blocks	Inserts a RAWBYTES block at a 64-byte-aligned address for every frame, plus a sprite pointer setup.
Save `.bin`	Writes 64 bytes per frame (raw sprite data without padding).

C64 Character ROM Browser ("Char map")

Read-only viewer of the C64 character ROM (the built-in PETSCII font). Open via the **C64 chargen** entry in the Toolkit menu. Useful for finding the screen-code of a glyph before writing it with `RAWBYTES` or `TEXT`.

Feature	Description
Two character sets	Tab 1: **Set 1 — Upper/Graphics** (default mode after power-on). Tab 2: **Set 2 — Lower/Upper** (after `$0E` switch).
Glyph grid	16×16 grid of all 256 characters. Click a glyph to see its detail panel: zoomed 8×8 pixel view, screen code (decimal + hex), PETSCII codes (both default and shifted), and the raw 8-byte bitmap.
Detail panel	Shows the selected glyph's screen code, PETSCII codes, and the eight raw bytes — ready to paste into a `RAWBYTES` or BYTE block.
Read-only	No editing here — use the Character Editor (below) to modify glyphs.

Character Editor (Charset)

256-character 8×8 charset editor. Open via Toolkit → Character Editor.

Feature	Description
Load ROM	Imports the C64 ROM charset directly from VICE's `chargen` (no file picker).
Load `.bin`	Imports an external 2048-byte charset binary.
Per-char preview	16-wide grid of all 256 glyphs with the current cell highlighted.
Pixel editor	8×8 single-character editor with toggle/invert/clear tools.
Export blocks	Appends RAWBYTES at $0800 (block 2) or $3800 (block 7) with the encoded charset.

Map Editor (Multilayer Tilemaps)

Layered tilemap editor for static scenery, sprite spawn maps, collision data, and similar. Open via Toolkit → Map Editor.

Feature	Description
Layers	Multiple named layers, each with its own tileset and opacity.
Brushes	Single-tile, fill, line and rectangle modes.
Clear menu	Per-layer or whole-map clear with confirmation.
Image import	Drop a PNG of a tilemap; the editor auto-slices into tiles.
Export blocks	Emits RAWBYTES blocks for tileset graphics + map data.

SID Editor (3-Voice Tracker)

Multi-instrument 3-voice tracker with a Web Audio preview engine. Open via Toolkit → SID Editor.

Per-instrument controls:

- Waveform checkboxes (TRI / SAW / PUL / NOI) — multiple waveforms can be ORed together.
- ADSR (attack / decay / sustain / release) shown as a drag-graph above the four sliders.
- Pulse width slider (0-4095) with optional ring/sync flags.
- Filter routing checkbox per voice; global filter cutoff / resonance / volume / mode (LP/BP/HP).

Tracker grid:

- 3 voices × up to 7 patterns × 32 rows = 7 × 32 = 224 rows max (8-bit row counter constrains it).
- Per-row: note + instrument index. Empty rows hold the previous note.
- Click-and-drag to paint a range of notes. Octave selector for the input note (0-7).
- Range copy / paste: select a span with shift-click, then **Copy** / **Paste** in the tracker toolbar.
- Speed slider sets the IRQ tick divisor (frames between rows).

Files menu exports:

Export	What it does
`Save .bin…`	Writes the editor's native serialized format (instruments + patterns + sequence).
`Export blocks (data only)`	Appends instrument table + pattern blocks to the program at `* = $C000`.
`Export blocks + miniplayer`	Adds the full player (sid_init / sid_irq / sid_play_row / sid_set_voice) plus PAL frequency tables. After export, place a `JSR sid_init` in your main code where the music should start.
`Export asm (clipboard)`	Copies the full assembly source to the clipboard.

Player ZP usage: `$FB` (tick counter), `$FC` (row index), `$FD` (set_voice temp). These conflict if your main code uses them — relocate via Expert mode if needed.

Known limits:

- Single linear pattern list (no per-voice sequence table yet).
- 8-bit row counter limits to 7 patterns × 32 rows.
- C64 `$D418` global volume is shared across voices — the per-instrument volume slider is informational; sustain level (`S` of ADSR) is the effective per-voice volume.
- Web Audio preview is approximate: PWM modulation, ring/sync, and the SID filter character differ from the real chip.